

Tema 6 - Aspectos avanzados

Francisco Gortázar

Type hints

- Son anotaciones que indican los tipos esperados de los argumentos y valor devuelto por una función
- Proporcionan información de tipos a los desarrolladores
- El intérprete los ignora:
 - No hay compilación, por lo que no se pueden verificar en tiempo de edición
 - En ejecución son ignorados y el intérprete no verifica si se pasa un valor del tipo correcto

```
def suma(a: int, b: int) -> int:  
    return a + b
```

Type hints

- Se pueden asignar tipos a variables también

```
a: int = 0  
b: bool
```

- También se puede hacer con colecciones:

```
names: list[str]  
prices: dict[str, float] = {"milk", 0.89}
```

Type alias

- Permiten dar un nombre nuevo a tipos ya existentes
- Útil para hacer el código más entendible
- Asignamos el alias con la palabra reservada **type** (Python 3.12+)

```
type Vector = list[float]

def scalar_product(scalar: float, v: Vector) -> float:
    return [scalar * x for x in v]

result = scale(2.0, [1.0, -4.2, 5.4])
```

Definición de funciones generadoras

- Cuando recorremos una colección, nuestro código es llamado por cada elemento de la misma:

```
for name in names:
    print(name)
```

- Esto es el resultado de invocar implícitamente una función generadora o iterador

Definición de funciones generadoras

- Una función normal se ejecuta de principio a fin y usa return para entregar un valor y finalizar
- Las funciones generadoras devuelven valores de una colección de elementos, pero uno cada vez que son llamadas:
 - Entrega un valor y pero pausa su ejecución guardando su estado para continuar justo donde se quedó la próxima vez que se le llame
 - Evaluación perezosa: no se genera un valor hasta que no se pide
- Las funciones generadoras no pueden usar return para devolver un valor
 - La llamada a la función finalizaría como una función normal
 - En su lugar deben usar la palabra reservada **yield** para devolver el siguiente valor

Definición de funciones generadoras

- Cuando llamas a una función generadora, no obtienes el resultado inmediatamente, sino un objeto generador
- Para obtener los valores, debes iterar sobre él (por ejemplo, con un bucle `for`).
- Ejemplo: fibonacci como función generadora

```
1 def fibonacci(n):
2     a, b = 0, 1
3     for _ in range(n):
4         yield a
5         a, b = b, a + b
```

Definición de funciones generadoras

- Uso:

```
for num in fibonacci(10):
    print(num)
```

Definición de funciones generadoras

- Las funciones generadoras permiten trabajar con colecciones de elementos sin tener que guardarlos todos en memoria:
- Ejemplo con listas:

```
def fibonacci(num_terminos):
    a, b = 0, 1
    lista_num_terminos = []
    while(num_terminos > 0):
        lista_num_terminos.append(a)
        a = b
        b = a + b
        num_terminos -= 1
    return lista_num_terminos
lista = fibonacci(100)
suma = sum(lista) # lista num terminos contiene los num_terminos
```

Definición de funciones generadoras

- Las funciones generadoras permiten trabajar con colecciones de elementos sin tener que guardarlos todos en memoria:
- Ejemplo con función generadora:

```
def fibonacci(n):  
    a, b = 0, 1  
    for _ in range(n):  
        yield a  
        a, b = b, a + b  
suma = sum(fibonacci(100)) # Sólo se mantiene un elemento en memoria cada vez
```

Definición de funciones generadoras

- Ejemplo:

```
it = fibonacci(10)  
print(type(it))
```

- Produce:

```
<class 'generator'>
```

Definición de funciones generadoras

- Internamente Python convierte la función en una clase con el método `__next__()` que permite obtener el siguiente elemento:

```
it = fibonacci(10)  
for i in range(10):  
    print(it.__next__())
```

Definición de funciones generadoras

- Una vez consumidos los elementos de la función generadora, tratar de obtener más valores lanza la excepción `StopIteration`:

```
for i in range(10):
    print(it.__next__())
print(it.__next__()) # Tratamos de imprimir un valor más
```

- Produce:

```
Traceback (most recent call last):
  File "/home/patxi/git/lp-python-2026/tema-6/ejem3-func-generadoras.py", line 16, in <module>
    print(it.__next__())
    ~~~~~^
StopIteration
```

Gestión de excepciones

- Python permite capturar los errores que puedan producirse durante la ejecución de nuestro código
- Estructura básica:

```
try:
    numero = int(input("Introduce un número: "))
    resultado = 10 / numero
    print(f"Resultado: {resultado}")
except ValueError:
    print("¡Error! Debes introducir un número válido.")
except ZeroDivisionError:
    print("¡Error! No puedes dividir entre cero.")
```

Gestión de excepciones

- Si es necesario podemos capturar la variable de error:

```
try:
    numero = int(input("Introduce un número: "))
    resultado = 10 / numero
    print(f"Resultado: {resultado}")
except ValueError e:
    print("¡Error! Debes introducir un número válido.")
    print(str(e))
except ZeroDivisionError e:
    print("¡Error! No puedes dividir entre cero.")
```

Gestión de excepciones

- Podemos definir acciones a ejecutar si no hay errores en un bloque else:

```
try:
    archivo = open("datos.txt", "r")
except FileNotFoundError:
    print("El archivo no existe.")
else:
    print("Archivo leído con éxito.")
    print(archivo.read())
```

Gestión de excepciones

- Podemos también definir acciones que deben ejecutarse siempre:

```
try:
    archivo = open("datos.txt", "r")
except FileNotFoundError:
    print("El archivo no existe.")
else:
    print("Archivo leído con éxito.")
    print(archivo.read())
finally:
    print("Cerrando recursos...")
    # Esto asegura que si el archivo se abrió, se cierre pase lo que pase
    if archivo != None and not archivo.closed:
        archivo.close()
```

Gestión de excepciones

- Podemos lanzar excepciones desde nuestro código si queremos indicar una situación de error:

```
def establecer_edad(edad):
    if edad < 0:
        raise ValueError("La edad no puede ser negativa.")
    print(f"Edad establecida en {edad}")

try:
    establecer_edad(-5)
```

```
except ValueError as e:
    print(f"Capturamos un error personalizado: {e}")
```

Más sobre argumentos posicionales y nombrados

- Una función en Python puede recibir más argumentos de los especificados en su definición
- Estos argumentos extra pueden ser capturados con la notación `*args` para parámetros posicionales y `**kwargs` para parámetros nombrados
- `*args` debe aparecer siempre antes de `**kwargs`
- Ejemplo 1:

```
def strange_func1(a, b, *args):
    print(f"a: {a}")
    print(f"b: {b}")
    for arg in args:
        print(f"arg: {arg}")

strange_func(1, 2, 3, 4, 5)
```

Más sobre argumentos posicionales y nombrados

- Ejemplo 2:

```
def strange_func2(a, b, **kwargs):
    print(f"a: {a}")
    print(f"b: {b}")
    for key, value in kwargs.items():
        print(f"{key}: {value}")

strange_func2(1, 2, c=3, d=4, e=5)
```

Más sobre argumentos posicionales y nombrados

- Ejemplo 3:

```
def strange_func3(a, b, *args, **kwargs):
    print(f"a: {a}")
    print(f"b: {b}")
    for arg in args:
        print(f"arg: {arg}")
    for key, value in kwargs.items():
        print(f"{key}: {value}")

strange_func3(1, 2, 3, 4, 5, c=6, d=7, e=8)
```

Más sobre argumentos posicionales y nombrados

- Si pasamos argumentos posicionales cuando se esperan argumentos nombrados o viceversa, obtenemos un error:

```
strange_func2(1, 2, 3, 4, 5)
```

- Produce:

```
TypeError: strange_func2() takes 2 positional arguments but 5 were given
```

Más sobre argumentos posicionales y nombrados

- Si tenemos una lista o un diccionario podemos pasar su contenido a una función:
 - Para listas usamos el operador `*` (argumentos posicionales)
 - Para diccionarios usamos el operador `**` (argumentos nombrados)
- Ejemplo:

```
numeros = [1, 2, 3, 4, 5]
cosas_nombradas = {"c": 6, "d": 7, "e": 8}

strange_func3(0, 0, *numeros, **cosas_nombradas)
```